

MIT 8.370.2x — COURSE-LEVEL QAI LEARNING EXTRACTION

Quantum Information Science I, Part II | SU1–SU4 Consolidated Knowledge Record

MIT Open Learning Library | QAI Knowledge Capture | 2026

Course status: SU1–SU4 completed and consolidated

1. COURSE IDENTITY & SCOPE

- Course: MIT Open Learning Library — 8.370.2x Quantum Information Science I, Part II.
- Consolidated subunits: SU1 Quantum teleportation and superdense coding; SU2 The Deutsch–Jozsa and Simon’s algorithms; SU3 The QFT and Shor’s quantum factoring algorithm; SU4 Quantum search and quantum simulation algorithms.
- Primary learning scope: entanglement and quantum communication; oracle-based algorithms; phase kickback and interference; hidden-structure discovery; QFT and phase estimation; order finding and Shor; Grover search; Hamiltonian simulation; quantum chemistry simulation; VQE/QPE; continuous-time search.
- QAI objective: extract reusable concepts, algorithmic patterns, data structures, primitives, control-plane ideas, compiler/runtime implications, resource models and validation pathways while clearly separating MIT course evidence from QAI architectural interpretation.

2. COURSE-LEVEL EXECUTIVE SUMMARY

Across SU1–SU4, the learning progresses from fundamental multi-qubit protocols to reusable quantum algorithms and simulation methods. The strongest common engineering pattern is a separation of logical intent, quantum execution, classical control/inference, and physical-resource realization.

The course provides a coherent foundation for the QAI stack: SU1 introduces entanglement resources, measurement, classical feed-forward and quantum-assisted communication; SU2 introduces oracle construction, phase kickback, interference, hidden-subgroup discovery and quantum-generated constraints; SU3 adds QFT, phase estimation, order finding and the quantum/classical factoring workflow; SU4 expands the model to amplitude amplification, Hamiltonian evolution, simulation, VQE and representation-aware execution.

The resulting QAI architectural direction is not to hard-code individual algorithms into the platform. Instead, the learning supports a layered abstraction in which algorithms are represented symbolically, compiled/decomposed according to backend capabilities, executed through a hybrid runtime, and governed by a control plane that manages policy, resources, measurement feedback, validation and retry/termination decisions.

3. COURSE LEARNING PROGRESSION

Subunit	Primary Concepts	Algorithmic Pattern	QAI Architectural Contribution
SU1	Bell states, teleportation, superdense coding, measurement, feed-forward, imperfect entanglement	Quantum operation + entangled resource + measurement + classical control	Entanglement resource model, correction policies, hybrid control boundary, state/resource addressing
SU2	Deutsch–Jozsa, Simon, Boolean promises, oracles, phase kickback, interference, GF(2) constraints	Quantum sampling → classical constraint extraction/inference	QAI Oracle, Operation, Circuit, Constraint and Algorithm abstractions; compiler synthesis
SU3	QFT, inverse QFT, QPE, eigenphases, modular exponentiation, order finding, Shor	Phase encoding → inverse QFT → measurement → classical reconstruction	Phase-estimation primitive candidate, hybrid runtime, precision/backend/retry control
SU4	Grover, reflections, Hamiltonian simulation, Trotterization, chemistry, VQE, continuous-time search	Amplitude amplification / Hamiltonian evolution / hybrid optimization	Search and Hamiltonian primitives, representation transformation, resource-aware execution

4. COURSE-LEVEL CORE LEARNINGS

- Entanglement is a reusable computational/communication resource, not merely a circuit state.
- Measurement is both a source of classical information and a control boundary that can determine subsequent quantum operations.
- Quantum algorithms frequently generate evidence, constraints, phases or amplitudes that require classical interpretation, reconstruction or optimization.
- Oracle construction is a recurring abstraction across Deutsch–Jozsa, Simon and Grover.
- Phase kickback and interference are recurring mechanisms for converting hidden structure into measurable information.
- QFT/inverse QFT provide a bridge from phase-domain information to computational-basis measurement.
- Quantum Phase Estimation connects controlled-unitary evolution to classical phase reconstruction and is a key bridge into order finding and Shor.

- Grover demonstrates that execution length is an algorithmic control variable: more iterations are not always better.
- Hamiltonian evolution provides a representation distinct from circuit execution, creating an opportunity to reason about equivalent computational intent across representations.
- VQE demonstrates a practical hybrid loop in which a classical optimizer and quantum subroutine cooperate.
- Logical operations must be separated from native-gate realization, backend topology, fidelity and resource constraints.
- Physical-qubit identity, logical-qubit identity, quantum-resource identity and state/subspace identity should not be conflated.
- Educational algorithms and QAI primitive candidates require independent implementation, simulator testing and hardware validation before production promotion.

5. COURSE-WIDE REUSABLE ALGORITHMIC PATTERNS

Pattern	Reusable Structure	Evidence
Quantum Operation + Classical Control	Quantum state → measurement → classical result → conditional quantum operation → continued execution.	SU1
Entanglement-Assisted Communication	Entangled resource → encode/measure → classical or quantum transfer → decode/correct.	SU1
Oracle → Phase Kickback → Interference	Encode a function into a reversible oracle, transform phase information, then measure an observable property.	SU2
Quantum Sampling → Classical Constraint Solving	Quantum execution produces measurement vectors/constraints; classical mathematics reconstructs hidden structure.	SU2
Phase Encoding → Inverse QFT → Classical Reconstruction	Controlled-U powers encode eigenphase; inverse QFT exposes binary phase information; classical processing reconstructs the target.	SU3
Amplitude Amplification	Prepare superposition → phase mark solutions → reflect → repeat for an optimized number of iterations.	SU4
Hamiltonian Decomposition → Approximate Evolution	Represent H as components → product-formula/Trotter decomposition → execute under accuracy/resource constraints.	SU4
Hybrid Quantum-Classical Optimization	Quantum subroutine → measurement → classical parameter update → repeat.	SU4
Logical Intent → Representation Transformation → Backend Mapping	Preserve computational meaning while selecting circuit/unitary/Hamiltonian/geometric representation and native execution path.	SU4/QAI interpretation

6. COURSE-LEVEL QAI STACK MAPPING

- QAI Language — first-class representations for qubits/logical qubits, quantum states, gates, circuits, oracles, entanglement resources, measurements, constraints, Hamiltonians, phase estimates and algorithm intent.
- QAI Compiler — logical-to-native gate synthesis, decomposition, circuit optimization, oracle/reflection synthesis, QFT construction, Hamiltonian decomposition and backend-aware mapping.
- QAI Runtime — controlled execution of quantum circuits, measurements, conditional operations, Hamiltonian evolution and coordination with classical computation.
- Hybrid Runtime — explicit quantum/classical workflow execution for teleportation feed-forward, Simon inference, Shor reconstruction, VQE and other hybrid loops.
- QAI Control Plane — execution policy, subscription/resource constraints, measurement handling, correction policy, iteration/time-step control, validation, retry budgets and backend selection.
- Quantum Control Plane — lower-level coherent execution, native control realization, logical-to-physical mapping, QEC/FTQC interfaces and hardware feedback.
- QAI OS — lifecycle and resource management for logical, virtual and physical quantum assets; proposed architectural direction.
- Resource Registry — entanglement resources, QPU/simulator resources, qubits, ancillas, circuits and Hamiltonian resources with capability/quality metadata.
- Capability Registry — native gates, synthesis rules, QFT/QPE, Hamiltonian simulation, oracle construction and backend capabilities.
- QAI Labs — simulator and hardware validation of candidate primitives and cross-course patterns.
- QAI Product Foundry — pathway from validated research primitives to reusable services/products; production promotion only after independent validation.

7. COURSE-LEVEL QAI DATA / OBJECT MODEL

- QAI_Operation — logical operation, targets, controls, parameters, reversibility/unitarity and metadata.
- QAI_GateFamily — primitive gates, synthesis/decomposition rules and backend support.
- QAI_Circuit — registers, operations, dependencies, measurements and execution metadata.
- QAI_Oracle — function definition, promise, registers and reversible implementation.
- QAI_State / QuantumState — basis/superposition representation, amplitudes, phases and register structure.
- EntanglementResource — resource identity, endpoints, state type, fidelity/quality, error probability, lifetime and availability.

- QAI_Constraint — measured vector, hidden variable, inner-product rule and classical constraint system.
- QAI_Hamiltonian — Hamiltonian terms, representation, simulation method and accuracy requirements.
- QAI_PhaseEstimate — phase bit string, estimated phase, precision and reconstruction metadata.
- QAI_Algorithm — prepare(), oracle(), transform(), measure(), infer(), verify() and related workflow stages.
- QAI_BackendProfile — native gates, topology, fidelity/error characteristics, mapping rules and resource capabilities.

8. COURSE-LEVEL CONTROL / RESOURCE MODEL

- Address Plane — physical-qubit ID, logical-qubit ID, resource ID, encoded-state identity and state/subspace reference.
- State Plane — computational basis, superposition, entanglement, encoded/logical states and Hamiltonian/state representations.
- Control Plane — measurement results, correction policies, constraints, iteration count, precision, retry budget, termination policy and routing policy.
- Execution Plane — gates, measurements, controlled operations, QFT/QPE, Hamiltonian evolution and backend mapping.
- Resource quality — fidelity, error probability, coherence/lifetime and availability should influence execution choices where supported by the implementation.
- Logical/physical separation — the course learning supports keeping logical algorithmic intent distinct from physical hardware realization and native gate constraints.

9. COURSE-LEVEL ALGORITHMS & QAI PRIMITIVE CANDIDATES

Primitive / Asset	Source	Purpose	Maturity
Teleport / SuperdenseEncode / BellMeasure	SU1	Communication and entanglement protocol primitives	Candidate
QAI_Oracle / QAI_Operation / QAI_Circuit	SU2/SU4	Reusable oracle and circuit abstractions	Strong candidate
QAI_Constraint / hidden-structure solver	SU2	Quantum-generated constraints + classical inference	Candidate
Phase Estimation / Quantum Phase Extraction	SU3	Controlled-U → inverse QFT → measurement → phase reconstruction	Educational candidate; validate
QAI_GroverSearch	SU4	Amplitude amplification search	Candidate
QAI_Reflection	SU4	Reflection/diffusion synthesis	Candidate
QAI_HamiltonianEvolution	SU4	Hamiltonian time evolution	Candidate
QAI_HamiltonianDecomposition	SU4	Trotter/product-formula decomposition	Candidate
QAI_RepresentationTransform	SU4/QAI	Circuit/unitary/Hamiltonian/state representation transformation	Research candidate
QAI_SearchExecutionPolicy	SU4/QAI	Optimal iteration/time-step and termination control	Research candidate
QAI_QuantumObjectLifecycle	SU4/QAI	Create → compose → manipulate → validate → execute → measure → destroy	Future capability

10. COURSE-WIDE ARCHITECTURAL INSIGHTS

- Abstraction over physical hardware: logical algorithms should not be coupled directly to a particular QPU's physical qubit IDs or native gate set.
- Measurement as a first-class event: measurement results can be data, constraints, corrections, routing metadata or termination evidence.
- Quantum/classical boundary as a designed interface: classical post-processing is part of the algorithmic workflow rather than an afterthought.
- Resource-aware execution: entanglement quality, backend capability, gate fidelity, precision, iteration count, Hamiltonian decomposition cost and available resources can affect execution decisions.
- Representation independence: where validated, the same computational intent may be represented as a circuit, unitary, Hamiltonian, state or geometric model.
- Validation-driven promotion: educational evidence supports learning and candidate design, but not production maturity.
- Lifecycle management: quantum objects require creation, composition, manipulation, validation, execution, measurement and eventual release/destruction.
- Hybrid architecture is intrinsic: the course repeatedly demonstrates that quantum computation and classical computation work together.

11. COURSE-LEVEL VALIDATION STRATEGY

- Reference implementations: implement Bell/teleportation/superdense coding, Deutsch–Jozsa, Simon, QFT, inverse QFT, QPE, small Shor, Grover and Hamiltonian/VQE examples.
- Simulator validation: reproduce the exact educational examples and extend to additional inputs, predicates, phases, N/M ratios and Hamiltonians.
- Noise validation: measure sensitivity to gate error, measurement error, decoherence and imperfect entanglement where applicable.

- Hardware/QPU validation: compare logical predictions with native-gate implementations and real backend constraints before making production claims.
- Cross-course validation: compare common oracle, phase-kickback, hidden-structure, phase-estimation, representation and hybrid-execution patterns.
- Benchmark dimensions: circuit depth, gate count, ancilla count, approximation error, success probability, runtime, shots, resource usage and retry rate.
- Acceptance criteria: reproducibility, functional equivalence within defined tolerance, documented resource profile, backend-aware implementation evidence and clearly stated limitations.
- FTQC progression: validate whether logical/virtual abstractions survive mapping to fault-tolerant logical operations and QEC interfaces in later study.

12. COURSE-LEVEL CROSS-SUBUNIT CONVERGENCE

Convergent Concept	SU Evidence	QAI Interpretation
Quantum → Classical → Quantum control loop	SU1, with classical feed-forward	Foundation for QAI hybrid control-plane execution
Oracle abstraction	SU2, SU4	Reusable QAI_Oracle and compiler synthesis capability
Phase kickback / interference	SU2, SU3	Common mechanism for hidden structure and phase estimation
Quantum measurement → classical inference	SU1–SU4	Hybrid Runtime + Intelligence Layer
QFT / phase transformation	SU3	Phase-estimation and spectral-analysis capability
Amplitude amplification / reflection	SU4	Search primitive + execution policy
Hamiltonian evolution	SU4	Simulation primitive + compiler decomposition
Logical vs physical realization	All; explicit in SU1/SU4	QAI Language/Compiler/Control Plane separation
Resource-aware execution	SU1 imperfect entanglement; SU3 precision/backend; SU4 iteration/resource constraints	Control-plane resource policy

13. COURSE-LEVEL QAI MATURITY ASSESSMENT

- Educational knowledge: High confidence across SU1–SU4; all four subunit records were completed.
- Algorithmic pattern extraction: High confidence for the patterns directly supported by course problems and lectures.
- QAI primitive candidates: Strong design input, but candidates remain subject to independent implementation and validation.
- Production capability: Not established by the course evidence alone; no claim of production QPU performance or deployed QAI service is made.
- Architecture value: High as a design/reference input because the four subunits collectively cover communication, hidden structure, phase estimation, search, simulation and hybrid execution.
- Next engineering step: convert the strongest recurring primitives into simulator-backed reference implementations and then validate them against suitable hardware/backends.

14. COURSE-LEVEL IMPLEMENTATION ROADMAP

1. Normalize the QAI Language object model: Operation, Circuit, Oracle, State, Constraint, Hamiltonian, PhaseEstimate and Resource.
2. Implement reusable simulator reference primitives for SU1–SU4.
3. Build compiler transformations for oracle synthesis, reflection synthesis, QFT/inverse QFT, Hamiltonian decomposition and native mapping.
4. Implement hybrid-runtime patterns for measurement feed-forward, classical inference, phase reconstruction, VQE optimization and search control.
5. Extend the QAI Control Plane with precision, shots, resource-quality, iteration/time-step, validation and retry policies.
6. Create Resource and Capability Registries for simulator/QPU/backend characteristics.
7. Validate across multiple backends and noise models.
8. Promote only independently validated primitives into the QAI Product Foundry / Enterprise Library.

15. COURSE COMPLETION / EVIDENCE

- SU1 completed: lecture concepts and problem-set evidence captured; teleportation and superdense-coding results accepted.
- SU2 completed: Deutsch–Jozsa and Simon lecture/problem-set learning captured, including hidden-period example.
- SU3 completed: QFT I–III and Phase Estimation I–III all completed correctly; the extraction records the six problem-set parts and graded results.
- SU4 completed: Grover and continuous-time quantum simulation problem-set learning captured and validated against the supplied grading evidence.
- Course-level record status: SU1–SU4 merged; QAI interpretations remain explicitly distinguished from MIT course requirements.

16. PROVENANCE & FIDELITY NOTE

This consolidated document is derived from the four user-provided subunit extraction records. The source records explicitly distinguish MIT course evidence from QAI architectural interpretations. This course-level synthesis preserves that distinction: statements about what the MIT material teaches are source-derived, while QAI primitives, control-plane mappings, product ideas,

resource models and implementation proposals are treated as architectural interpretations/design inputs rather than MIT-prescribed implementations.

PART II — CONSOLIDATED SUBUNIT RECORDS

Full source-derived SU1–SU4 extraction records retained below for traceability and future reuse.

SU1 — Quantum teleportation and superdense coding

Source-derived subunit extraction record retained as part of the course-level master document.

SU2 — The Deutsch–Jozsa and Simon’s algorithms

Source-derived subunit extraction record retained as part of the course-level master document.

SU3 — The QFT and Shor’s quantum factoring algorithm

Source-derived subunit extraction record retained as part of the course-level master document.

SU4 — Quantum search and quantum simulation algorithms

Full source-derived SU4 extraction record retained for traceability and future reuse.

MIT OLL Standard Course Extraction

8.370.2x — SU1: Quantum teleportation and superdense coding

QAI Knowledge Capture / Module Record

1. Course Identity

Course	MIT Open Learning Library — 8.370.2x Quantum Information Science I, Part II
Module / SU	SU1: Quantum teleportation and superdense coding
Primary scope	Bell states, Bell-basis measurement, quantum teleportation, superdense coding, entanglement, classical feed-forward, quantum-assisted communication, imperfect entanglement, and QAI control-plane implications.
Capture status	Completed module knowledge capture based on lecture questions and problem-set discussion completed in this session.
Granularity	Module/SU is the primary extraction unit; lecture, question, and problem-set items remain granular evidence units.

2. Fidelity / Source Structure

This record preserves the terminology and problem structures encountered in the MIT OLL material and separately records QAI-oriented interpretations developed during the discussion. QAI mappings are architectural interpretations, not claims that the MIT course itself specifies a QAI implementation.

- Lecture/concept evidence: Bell-basis measurement, teleportation circuit, superdense coding.
- Problem-set evidence: quantum teleportation; Superdense Coding 1/3, 2/3, 3/3.
- Discussion evidence: probability vs expected value, classical feed-forward, correction operators, Clifford/non-Clifford generality, resource quality, logical vs physical error, and quantum state-space addressing.

3. Module Map

Lecture/content	Bell basis measurement; teleportation circuit; superdense coding; Bell-state encoding/decoding.
Problem sets	Quantum teleportation; Superdense Coding (3-part problem set).
Core mathematical objects	Bell states, Pauli operators I/X/Y/Z, measurement outcomes, conditional corrections, entangled resources.
Core protocol pattern	Quantum operation + entanglement resource + measurement + classical control + conditional quantum operation.

4. Lecture / Content Records

4.1 Bell Basis Measurement

The Bell measurement circuit converts Bell-basis states into computational-basis measurement outcomes. The reusable pattern is CNOT followed by Hadamard and computational measurement.

- Bell states form an orthonormal basis for two-qubit states.
- Measurement produces a classical two-bit label.
- The result can become control metadata for a later quantum operation.

4.2 Quantum Teleportation

Teleportation transfers an unknown quantum state using a shared entangled resource and classical communication. The unknown quantum state is not transmitted through the classical channel.

- Alice combines the unknown state with her entangled qubit and performs a Bell measurement.
- The measurement generates two classical bits.
- Bob uses those bits to select a conditional correction.
- For the modified Ψ^- resource used in the problem, the accepted correction map was $00 \rightarrow Y$, $01 \rightarrow Z$, $10 \rightarrow X$, $11 \rightarrow I$.

4.3 Classical Feed-Forward

Measurement results are classical control information that select subsequent quantum operations. This is a fundamental hybrid quantum/classical control boundary.

Pattern: Quantum state $\rightarrow$ measurement $\rightarrow$ classical result $\rightarrow$ conditional quantum operation $\rightarrow$ quantum state

4.4 Superdense Coding

Superdense coding uses a shared entangled pair so Alice can encode two classical bits into one transmitted qubit. Bob performs a Bell measurement to recover the two classical bits.

- The modified protocol uses I , X , Z and Y as the four encoding choices.
- Alice's operation maps the shared entangled resource into distinguishable Bell states.
- Superdense coding transmits classical information; it is not a protocol for arbitrary quantum-state transmission.

5. Solved Questions / Evidence Record

Bell basis measurement circuit: Completed H/I placements around the CNOT so the four Bell states map to the specified classical outputs.

Teleportation circuit: Completed the teleportation structure: entanglement preparation, Bell measurement, classical outputs, and conditional correction.

Superdense coding with three qubits: Confirmed that the stated GHZ resource permits two classical bits to be conveyed by sending Alice's single qubit.

Quantum teleportation problem: For the modified Ψ^- resource, obtained Bob's state for result 00 and the correction map $00 \rightarrow Y$, $01 \rightarrow Z$, $10 \rightarrow X$, $11 \rightarrow I$.

Bell measurement probability: For a state expressed in the Bell basis, the probability of a specified Bell outcome is the squared magnitude of its coefficient; for 01 , $|c|^2$.

Superdense Coding 1 of 3: Computed the states resulting from I , X , Z and Y acting on Ψ^- .

Superdense Coding 2 of 3: Completed the encoding circuit using Y followed by classically controlled Z and X .

Superdense Coding 3 of 3: For the specified imperfect shared entangled state, the failure probability to leading order is $\epsilon^2/4$.

6. Mathematical Models / Representations

Bell states:

- $|\Phi^+\rangle = (|00\rangle + |11\rangle) / \sqrt{2}$
- $|\Phi^-\rangle = (|00\rangle - |11\rangle) / \sqrt{2}$

- $|\Psi+\rangle = (|01\rangle + |10\rangle) / \sqrt{2}$
- $|\Psi-\rangle = (|01\rangle - |10\rangle) / \sqrt{2}$

Teleport(state, resource, destination, correction_policy)

SuperdenseEncode(message_bits, entanglement_resource, encoding_map)

Specific imperfect-resource result: $P_{\text{fail}} \approx \varepsilon^2/4$.

7. Algorithms / Protocols

1. Bell measurement: apply CNOT → apply H → measure computational basis.
2. Teleportation: shared entanglement → Bell measurement → transmit two classical results → select correction → reconstruct the state.
3. Superdense encoding: shared Bell resource → select I/X/Y/Z from two classical bits → transform Bell state → transmit one qubit → Bell measurement.
4. Resource-aware communication: evaluate entanglement quality → select/replace resource → execute protocol → evaluate reliability.

8. Reusable Patterns

Bell Measurement: Entangled state → CNOT → H → measurement → classical label.

Teleportation: Quantum state + entangled resource → Bell measurement → classical control → correction → reconstructed state.

Superdense Coding: Classical bits + entangled resource → gate selection → encoded Bell state → one-qubit transmission → Bell decoding.

Classical Feed-Forward: Measurement → classical result → conditional quantum operation.

Hybrid Control Loop: Quantum → measurement → classical decision → quantum operation → continued execution.

Resource-Aware Communication: Resource selection → quality/fidelity evaluation → protocol execution → reliability evaluation.

9. Candidate QAI Language Primitives

- Qubit, LogicalQubit, QuantumState, BellState, EntanglementResource
- QuantumOperation, Gate, Measurement, ControlledOperation, ConditionalOperation, CompositeOperation
- Teleport, SuperdenseEncode, BellMeasure, BellDecode
- ClassicalBit, MeasurementResult, Correction, CorrectionMap, PauliFrame
- QuantumChannel, ClassicalChannel, HybridChannel
- Fidelity, ErrorRate, LogicalError, ProtocolReliability
- QuantumResource, QuantumAddress, StateSpace, Subspace

10. QAI Stack Mapping

Bell states	Entanglement resource model and reusable two-qubit state basis.
Bell measurement	Quantum measurement primitive and quantum-to-classical label conversion.
Teleportation	Quantum-state transfer protocol.
Superdense coding	Quantum-assisted classical encoder/decoder.
I/X/Y/Z	Operator/gate abstraction; should not limit the generic QAI operation model.
Classical feed-forward	Hybrid quantum/classical control-plane mechanism.
Correction map	Policy/decision mapping from measurement metadata to

	quantum operations.
Imperfect entanglement	Resource quality/fidelity management and protocol reliability.
Physical vs logical error	Separate physical-resource metrics from logical-resource and protocol-level reliability.
State-space addressing	Abstraction beyond physical-qubit IDs: logical states, subspaces, encoded states, entanglement resources.

11. Operator / Gate Model for QAI

The standard examples use Pauli corrections, but the QAI language should expose a generic operation abstraction. S is a Clifford gate; T is a non-Clifford gate. The language should therefore support both categories without hard-coding teleportation to Pauli-only correction.

- QuantumOperation
- Gate → Pauli / Clifford / Non-Clifford
- Measurement
- ControlledOperation
- ConditionalOperation
- CompositeOperation

12. Quantum State-Space Addressing

A key architectural insight is that physical-qubit addressing may not remain the fundamental abstraction as systems scale. For n qubits there are 2^n computational basis states, while a general state is a superposition over those basis states.

Therefore distinguish: physical-qubit address $\neq$ logical-resource identity $\neq$ quantum-state description $\neq$ physical location.

- Physical-qubit ID
- Logical-qubit ID
- Entanglement/resource ID
- Encoded-state or code-space identity
- Subspace/state-space reference

13. QAI Switch / Router Implications

Superdense coding suggests a quantum-assisted encoding/multiplexing pattern, but it is not itself a conventional classical multiplexer. The deeper problem is addressing and routing quantum resources and state-space structures.

- Address Plane: physical ID, logical ID, resource ID, state/subspace reference.
- State Plane: basis states, superposition, entangled states, encoded/logical states.
- Control Plane: measurement results, correction policy, syndrome/control metadata, routing policy.
- Execution Plane: gates, measurements, teleportation, backend mapping.

14. Entanglement Resource Model

Candidate QAI resource metadata:

- resource_id
- state_type
- endpoint_A / endpoint_B
- fidelity / quality
- error_probability
- lifetime
- physical/logical classification

- availability

This enables resource-aware control decisions such as replacing a degraded resource, switching protocols, or invoking error-correction mechanisms. These are QAI architectural extensions derived from the learning, not MIT-prescribed implementation requirements.

15. Physical vs Logical vs Protocol Reliability

The $\varepsilon^2/4$ result belongs to the specific imperfect entangled-state model and should not be treated as a universal QEC scaling law.

- Physical layer: gate, measurement, decoherence, transmission and physical-qubit errors.
- Logical layer: residual errors after encoding/QEC.
- Protocol layer: success/failure probability of teleportation or superdense coding.
- Application layer: effect of protocol reliability on the final use case.

16. Cross-Course Convergence / Reuse

- Builds on Bell states, Pauli matrices, tensor products, measurement probabilities and unitary operations from 8.370.1x.
- Extends earlier matrix/circuit understanding into multi-step quantum protocols.
- Provides a foundation for QEC through measurement → classical syndrome/control → correction patterns.
- Provides communication primitives relevant to future QAI networking, routing, switching and hybrid channels.
- Demonstrates separation of quantum data, classical control and execution resources.

17. Reuse / Implementation Candidates

- QAI language quantum-protocol primitives.
- QAI Runtime conditional-operation and feed-forward engine.
- QAI Control Plane correction-policy engine.
- Entanglement resource registry and quality metadata.
- Quantum-assisted encoder/decoder components.
- QAI Switch/Router state-resource addressing model.
- Hybrid quantum/classical communication channel abstractions.

18. Validation / Evidence

- Lecture concept questions completed and checked in the MIT OLL interface.
- Quantum teleportation problem-set calculations completed and accepted.
- Superdense Coding 1/3 completed and accepted.
- Superdense Coding 2/3 completed and accepted.
- Superdense Coding 3/3 completed and accepted.
- Key mathematical results were checked against the course's displayed grading results.

19. Provenance

Primary source	MIT Open Learning Library — 8.370.2x Quantum Information Science I, Part II, SU1: Quantum teleportation and superdense coding.
Evidence type	User-provided screenshots, question text, solved answers, and discussion from the course session.
QAI interpretation	Derived during the working discussion and explicitly distinguished from MIT course requirements.
Related prior knowledge	Earlier completed MIT 8.370.1x SU1–SU3 knowledge capture, especially Bell states, Pauli matrices, tensor products, unitary operations, measurement and quantum circuits.

20. Module Summary

SU1 moves the learning progression from individual quantum gates and states toward complete quantum protocols. The central reusable abstraction is: Quantum Operation + Classical Control + Entangled Resource. Teleportation demonstrates quantum-state transfer using entanglement and classical feed-forward; superdense coding demonstrates entanglement-assisted classical information transmission. Together they provide a foundation for QAI language primitives, hybrid control-plane mechanisms, resource-aware quantum communication, and future QAI switching/routing abstractions.

21. Status

Module status	COMPLETE
Course status	8.370.2x in progress — SU1 completed.
Template status	Standard extraction record populated.
QAI stack status	Concepts, patterns, algorithms/protocols, candidate primitives, control-plane mappings, resource model and networking implications captured.

End of SU1 Knowledge Capture

8.370.2x Quantum Information Science I, Part II

SU2: The Deutsch–Jozsa and Simon’s Algorithms

Standard Learning & QAI Stack Template

1. Module Overview

SU2 develops the Deutsch–Jozsa and Simon quantum algorithms, emphasizing Boolean-function promises, reversible/permutation logic, universal gate families, Hadamard transforms, oracle construction, phase kickback, interference, hidden periodicity, measurement-derived constraints, and classical post-processing.

The central engineering lesson is a reusable algorithmic pattern: prepare a state, apply an oracle or transformation, encode/interfere phase information, measure, and use the result for classical inference.

2. Core Learning Objectives

- Distinguish constant and balanced Boolean functions and understand promise problems.
- Understand the Deutsch–Jozsa oracle and phase-kickback mechanism.
- Understand Simon’s promise and hidden XOR period s .
- Represent reversible classical logic using permutation matrices, CNOT and Toffoli.
- Understand universal gate families and gate synthesis/decomposition.
- Understand $H^{\otimes N}$ as a compact circuit representation of an exponentially large state-space transformation.
- Track amplitudes, signs/phases, interference and measurement probabilities.
- Understand measurement results as constraints that can be solved classically.
- Recognize Simon’s problem as a canonical Abelian Hidden Subgroup Problem.

3. Key Mathematical Models

- Hadamard: $H = (1/\sqrt{2})[[1,1],[1,-1]]$.
- N -qubit Hadamard layer: $H^{\otimes N}$.
- For a computational-basis input, $H^{\otimes N}$ creates a superposition over 2^N basis states with equal-magnitude amplitudes and input-dependent $\pm$ signs.
- Oracle: $U_f|x\rangle|y\rangle = |x\rangle|y \oplus f(x)\rangle$.
- With the work qubit in $|-\rangle$, oracle action produces phase kickback: $f(x)$ is encoded in the phase.

- Simon promise: $f(x)=f(y)$ iff $y=x\oplus s$.
- Simon constraint: $z\cdot s=0$ over $GF(2)$.
- Simon hidden subgroup: $G=(\mathbb{Z}_2)^n$ and $H=\{0,s\}$.

4. Quantum Circuit Primitives

- H — superposition and interference.
- X, Y, Z — single-qubit transformations.
- S and T — phase gates and universal-gate synthesis vocabulary.
- CNOT — controlled reversible transformation and oracle primitive.
- Toffoli — reversible Boolean computation and bridge between classical reversible logic and quantum circuits.
- I — explicit pass-through/no-op primitive.
- Tensor products — multi-qubit operators and states.
- Oracle U_f — algorithm-specific reversible function encoding.

5. Universal Gate Families & Synthesis

The lecture question established these valid synthesis relationships:

- X can be created from CNOT with suitable fixed/control configuration.
- Toffoli can be synthesized from H, T and CNOT.
- CNOT can be constructed from Toffoli with suitable fixed/control inputs.

QAI pattern: logical operation $\rightarrow$ gate-family resolution $\rightarrow$ synthesis/decomposition $\rightarrow$ optimization $\rightarrow$ native-gate mapping $\rightarrow$ execution.

6. Deutsch–Jozsa Algorithm

5. Prepare n data qubits in $|0\rangle$ and a work qubit in $|-\rangle$.
6. Apply $H\otimes n$ to the data register.
7. Apply U_f .
8. Use phase kickback to encode $f(x)$ into relative phases.
9. Apply a second $H\otimes n$ to convert phase information into measurable amplitudes.
10. Measure the data register and interpret the result under the stated promise.

Worked $n=2$ cases showed $j=00$ as constant, giving constructive interference at $|00\rangle$ and probability 1, while $j=01$ was balanced, producing the phase pattern $+, -, +, -$ and zero probability for $|00\rangle$.

7. Simon's Algorithm

11. Prepare two n -qubit registers in $|0\rangle$.
12. Apply $H\otimes n$ to the upper register.
13. Apply the Simon oracle U_f .
14. Measure/discard the lower register while retaining the induced state information in the upper register.
15. Apply $H\otimes n$ to the upper register and measure z .
16. Interpret each result as $z\cdot s=0$ over $GF(2)$.
17. Repeat to obtain independent constraints.
18. Solve the binary linear system for s .
19. Verify using $f(s)=f(0^n)$.

The worked two-bit example emphasized that z is the measured constraint vector, not the hidden period itself. For the solved example $z1=01$ led to the nontrivial solution $s=10$, verified by $f(10)=f(00)$.

8. Problem-Set Learning Captured

- Deutsch–Jozsa I: $n=2$ constant case, oracle state and final measurement probability.
- Deutsch–Jozsa II: $n=2$ balanced case, phase pattern and destructive interference.
- Simon I: two-bit permutation/oracle construction and hidden structure.
- Simon II: conditional state after measuring B, final Hadamards, possible z values, GF(2) constraint solving and verification.
- Exercises reinforced amplitude/phase tracking, tensor products, measurement, binary inner products and classical post-processing.

9. Reusable Algorithmic Patterns

Oracle Pattern: Encode a classical/hidden function into a reversible quantum transformation.

Phase-Kickback Pattern: Convert function values into relative phases.

Interference-to-Property Pattern: Convert phase structure into a measurable global property.

Quantum-Sampling-to-Constraint Pattern: Use measurements to generate mathematical constraints.

Quantum + Classical Solver Pattern: Quantum execution generates information; classical inference reconstructs hidden structure.

Hidden-Structure Discovery Pattern: Oracle $\rightarrow$ interference $\rightarrow$ measurement $\rightarrow$ constraints $\rightarrow$ hidden period/subgroup.

Gate-Synthesis Pattern: Logical gate $\rightarrow$ decomposition $\rightarrow$ optimization $\rightarrow$ native hardware operations.

Compact-State-Space Pattern: Represent exponentially large transformations symbolically instead of materializing full matrices.

10. QAI Language Data Structures

- QAI_Operation — logical operation, type, targets, controls, parameters, reversibility/unitarity and metadata.
- QAI_GateFamily — primitive gates, synthesis rules, decompositions and backend support.
- QAI_Circuit — qubits/registers, operations, dependencies, measurements and execution metadata.
- QAI_Oracle — function definition, registers, promise and reversible implementation.
- QAI_State — basis/superposition representation, amplitudes, phases and register structure.
- QAI_Constraint — measured z , hidden variable s , inner-product rule and GF(2) system.
- QAI_Algorithm — prepare(), oracle(), transform(), measure(), infer(), verify().
- QAI_BackendProfile — native gates, topology, fidelity/error characteristics and mapping rules.

11. QAI Stack Mapping

- QAI Language — symbolic states, gates, oracles, algorithms and constraints.
- QAI Compiler — gate synthesis, decomposition, circuit optimization and logical-to-native mapping.
- QAI Runtime — quantum execution and coordination with classical computation.
- QAI Control Plane — execution policy, measurement handling, post-processing and algorithm orchestration.
- QAI Algorithm Layer — reusable oracle/interference/hidden-structure templates.
- QAI Intelligence Layer — classical inference applied to quantum-generated measurements and constraints.
- QAI Hybrid Layer — coordinated classical and quantum execution.

12. Engineering Insights

- Do not confuse state-space size 2^N with an individual basis-state address such as 1207.
- Do not materialize a $2^N \times 2^N$ operator when a compact circuit/tensor representation is sufficient.
- Separate logical operations from physical/native gate realization.
- Treat measurements as evidence or constraints when solving hidden-structure problems.
- Keep quantum execution and classical inference distinct but coordinated.

- Build reusable algorithmic skeletons instead of isolated hard-coded circuits.
- Represent gate metadata and synthesis capability explicitly for QAI compiler/control decisions.

13. Hidden Subgroup Connection

Simon's problem is a canonical Abelian Hidden Subgroup Problem. The group is $(\mathbb{Z}_2)^n$ and the hidden subgroup is $H=\{0,s\}$. Quantum measurements provide vectors orthogonal to the hidden period; classical $\text{GF}(2)$ linear algebra reconstructs s . This provides a conceptual bridge toward the broader hidden-structure family, including the Abelian HSP formulation underlying Shor-type algorithms.

14. QAI Knowledge-Graph / Taxonomy Candidates

- Quantum Algorithms → Oracle Algorithms → Deutsch–Jozsa
- Quantum Algorithms → Hidden-Structure Algorithms → Simon
- Quantum Algorithms → Hidden Subgroup Problem → Abelian HSP
- Quantum Circuits → Interference → Hadamard Transform
- Quantum Circuits → Oracles → Reversible Function Encoding
- Quantum Circuits → Gate Synthesis → Universal Gate Families
- Quantum Measurement → Constraint Extraction → Classical Inference
- QAI Compiler → Logical-to-Native Mapping
- QAI Language → State / Operator / Oracle / Constraint Data Structures

15. Completion Status

SU2: The Deutsch–Jozsa and Simon's algorithms — completed. The lecture-based questions and worked problem-set questions covered in the study session were successfully completed, including the Simon hidden-period example.

16. Future Reading / Course Reference

MIT Open Learning Library — MITx 8.370.2x, Quantum Information Science I, Part II; SU2: The Deutsch–Jozsa and Simon's algorithms.

MIT Open Learning Library

QAI Learning Record | 8.370.2x | SU3

1. Course Identity

Course title: 8.370.2x Quantum Information Science I, Part II

MITx / course ID: MITx 8.370.2x

Term / course version: Course version/term not supplied in the inspected evidence package.

MIT Open Learning Library URL: Not supplied in the inspected evidence package.

Primary domain: Quantum information science; quantum algorithms; quantum Fourier transform; phase estimation; quantum factoring.

Course purpose: SU3 studies the QFT and Shor's quantum factoring algorithm, including quantum order/period finding, modular exponentiation, phase estimation, and the quantum/classical steps used for factoring.

Source material available: SU3 lecture concept questions/screenshots and the completed SU3 problem-set questions were supplied in the study session; the standard extraction template, extraction instructions, and completed SU2 sample were also supplied.

Extraction status: Completed for SU3 evidence reviewed in this study session. No SU3 notebook/code asset was supplied or inspected.

2. Course Fidelity and Source Structure

Preserve the actual MIT OLL organization. Do not replace MIT terminology with QAI terminology at this layer.

- Subunits / modules: SU3 — The QFT and Shor's quantum factoring algorithm (current evidence package).
- Lectures: SU3 concept-question material covering order finding, Shor's factoring structure, modular exponentiation, and factoring using phase estimation.
- Problem sets: Quantum Fourier Transform I–III; Phase Estimation I–III.
- Assessments / graded questions: 4 lecture concept questions and 6 problem-set parts were completed in the study session.
- References: MIT OLL course material as supplied; Wikipedia links appeared in one lecture concept-question prompt but were not used as primary extraction evidence.
- Other course components: No notebook, executable code, or independent hardware/QPU experiment was supplied for SU3.

3. Course Map

Subunit	Item	Type	Questions	Source Available	Status
SU3	QFT and Shor's quantum factoring algorithm	Subunit / module	—	Yes	Completed
SU3	Lecture concept questions: order finding, Shor, modular exponentiation, phase estimation	Assessment / graded questions	4	Yes	Completed
SU3	Quantum Fourier Transform I–III	Problem set	3	Yes	Completed
SU3	Phase Estimation I–III	Problem set	3	Yes	Completed
SU3	Solved questions and reasoning captured from study session	Solved questions	6 problem-set parts + 4 lecture questions	Yes	Completed

4. Module / Subunit Record

Subunit ID / name: SU3 — The QFT and Shor's quantum factoring algorithm

MIT source URL: Not supplied in the inspected evidence package.

Learning objectives: Understand QFT as a quantum analogue of the discrete Fourier transform; derive small QFT examples; express QFT outputs as tensor products; understand inverse-QFT circuit structure; understand phase estimation and controlled-U construction; connect eigenphases to order/period finding; understand the modular arithmetic and classical post-processing used by Shor.

Lectures / topics: Quantum order finding and Simon's algorithm; setting and structure of Shor's quantum factoring algorithm; modular exponentiation circuit; factoring using phase estimation.

Problem sets: [Enter source-supported information]

Questions: 4 lecture concept questions and 6 problem-set parts were completed. Problem-set score across the six parts: 10/10.

Important concepts: Period/order r ; modular exponentiation $f(x)=a^x \bmod N$; reversible modular multiplication; QFT and inverse QFT; tensor-product decomposition; controlled powers $U^{(2^m)}$; eigenvalues/eigenphases; phase estimation; continued fractions; classical GCD post-processing; probabilistic/retry behavior of Shor.

References: [Enter source-supported information]

5. Lecture / Content Record

Lecture / item ID: SU3 lecture content / concept-question set

Title: The QFT and Shor's quantum factoring algorithm

Type: Lecture / concept-question evidence

MIT source URL: Not supplied in the inspected evidence package.

Question count: 6 graded parts; 10/10 total.

Concepts covered: Order finding; Simon versus ordinary additive/modular periodicity; Shor's factoring setup; random choice of a ; odd co-prime factor assumption; modular multiplication and reversible arithmetic; efficient modular exponentiation; phase estimation; $U|x\rangle = |ax \bmod N\rangle$; continued-fraction and GCD post-processing.

Key mathematical ideas: r is the multiplicative order satisfying $a^r \equiv 1 \pmod{N}$ when $\gcd(a, N)=1$; $f(x)=a^x \bmod N$ is periodic; QPE estimates eigenphases; QFT/inverse QFT converts phase information to measurable computational-basis information; continued fractions reconstruct a candidate r .

Implementation relevance: Hybrid quantum-classical implementation pattern: reversible modular arithmetic and controlled- U operations on the quantum side, followed by measurement and classical reconstruction/validation. Educational evidence only.

6. Problem Set Record

Problem set ID / title: SU3 Problem Set — Quantum Fourier Transform I–III and Phase Estimation I–III

MIT source URL: Not supplied in the inspected evidence package.

Question count: [Enter source-supported information]

Original problem available: Yes — supplied as screenshots in the study session.

Solved questions available: Yes — all 6 parts were solved and checked as correct.

Notebook / code available: No notebook or executable code supplied.

Related lecture(s): QFT and Shor's factoring algorithm; phase estimation and its use in order finding.

7. Solved Question Extraction

Use one record per question and preserve the original problem/solution relationship.

Question ID: [Enter source-supported information]

Problem statement: [Enter source-supported information]

Given information: [Enter source-supported information]

Required result: [Enter source-supported information]

Solution / reasoning: [Enter source-supported information]

Mathematical derivation: [Enter source-supported information]

Algorithm — Yes — QFT, QPE, order finding and Shor workflow.

Key insight: [Enter source-supported information]

Assumptions: [Enter source-supported information]

Complexity: Lecture evidence states modular exponentiation can be implemented with $O(\text{poly}(\log N))$ elementary operations.
No detailed QFT/QPE gate-count benchmark was supplied.

Implementation clues: [Enter source-supported information]

Validation / result: [Enter source-supported information]

Solved Question Overview

Evidence scope: QFT I–III and Phase Estimation I–III.

Result: All six problem-set parts completed correctly.

Method: Direct mathematical derivation from supplied definitions and circuit descriptions.

QFT I

Question ID: QFT I

Problem statement: Compute $U_{\text{QFT}}|01\rangle$; compute $U_{\text{QFT}}[(|00\rangle + |11\rangle)/\sqrt{2}]$; compute $U_{\text{QFT}}|+++ \rangle$, choosing global phase so the $|00\rangle$ coefficient is real.

Given information: $n=2$ QFT definition; $|01\rangle = |1\rangle$; linearity; $|+++ \rangle$ is the QFT of $|000\rangle$ for three qubits.

Required result: The three transformed states.

Solution / reasoning: Apply the QFT sum directly; use phases $1, i, -1, -i$ for $j=1$; use linearity for the superposition; use $\text{QFT}^\dagger \text{QFT} = I$ for the final case.

Mathematical derivation: For $|01\rangle$, $e^{(2\pi i k/4)}$ gives $1, i, -1, -i$, yielding $1/2(|00\rangle + i|01\rangle - |10\rangle - i|11\rangle)$. For the superposition, the $|10\rangle$ terms cancel. Since $\text{QFT}|000\rangle = |+++ \rangle$, $\text{QFT}^\dagger|+++ \rangle = |000\rangle$.

Algorithm / procedure: Direct QFT evaluation; linearity; inverse-transform identity.

Key insight: QFT converts basis information into amplitudes/phases; interference can cancel selected basis amplitudes.

Assumptions: Exact 2- and 3-qubit arithmetic.

Complexity: Educational mathematical derivation; no implementation benchmark supplied.

Implementation clues: Symbolic/circuit-level implementation evidence only; no executable code or hardware result supplied.

Validation / result: 3/3 graded correct.

QFT II

Question ID: QFT II

Problem statement: Express $U_{\text{QFT}}|j\rangle$ in tensor-product form and find the m -th single-qubit state $|\phi_m\rangle$.

Given information: j is an n -bit string; k is represented in binary; QFT phase is $e^{(2\pi i j k/2^n)}$.

Required result: The m-th factor in the tensor product.

Solution / reasoning: Factor the phase using the binary expansion of k.

Mathematical derivation: $e^{(2\pi ijk/2^n)} = \prod_m e^{(2\pi i j k_m/2^m)}$, so each bit contributes $(1/\sqrt{2})(|0\rangle + e^{(2\pi i j/2^m)}|1\rangle)$.

Algorithm / procedure: Binary decomposition and tensor-product factorization.

Key insight: The QFT sum can be represented as a product of single-qubit phase states, supporting efficient circuit construction.

Assumptions: Exact symbolic derivation.

Complexity: Educational mathematical evidence; no gate-count experiment supplied.

Implementation clues: Symbolic/circuit-level implementation evidence only; no executable code or hardware result supplied.

Validation / result: 1/1 graded correct.

QFT III

Question ID: QFT III

Problem statement: Complete the two-qubit inverse-QFT circuit.

Given information: $n=2$ inverse QFT; supplied circuit with missing gates and a gate palette.

Required result: The missing gates in the supplied circuit.

Solution / reasoning: Use the inverse-QFT structure with Hadamards, conjugate phase rotation $S^\dagger$, and the controlled operations shown in the exercise.

Mathematical derivation: The completed circuit uses the gates shown in the graded screenshot: X placeholders/controls as represented by the exercise, together with H and $S^\dagger$.

Algorithm / procedure: Circuit-level decomposition of inverse QFT.

Key insight: Demonstrates how the mathematical inverse QFT is realized with elementary circuit operations.

Assumptions: Exact two-qubit circuit.

Complexity: No hardware execution evidence supplied.

Implementation clues: Symbolic/circuit-level implementation evidence only; no executable code or hardware result supplied.

Validation / result: 1/1 graded correct.

Phase Estimation I

Question ID: Phase Estimation I

Problem statement: Given $U|\psi_j\rangle = e^{(2\pi i\phi_j)}|\psi_j\rangle$ and $\phi_j = \psi_j/2^t$, find the state of the m-th qubit in the first register before the inverse QFT.

Given information: Controlled- $U^{(2^t(t-m))}$ operations; eigenstate $|\psi_j\rangle$; $\phi_j = \psi_j/2^t$.

Required result: The m-th first-register qubit state.

Solution / reasoning: Track the controlled power of U acting on the eigenstate.

Mathematical derivation: $U^{(2^t(t-m))}|\psi_j\rangle = e^{(2\pi i 2^t(t-m)\phi_j)}|\psi_j\rangle = e^{(2\pi i \psi_j/2^m)}|\psi_j\rangle$, giving $(1/\sqrt{2})(|0\rangle + e^{(2\pi i \psi_j/2^m)}|1\rangle)$.

Algorithm / procedure: Controlled phase kickback / eigenphase encoding.

Key insight: QPE encodes the eigenphase into the relative phases of the first-register qubits.

Assumptions: Exact t-bit phase representation.

Complexity: No implementation environment supplied.

Implementation clues: Symbolic/circuit-level implementation evidence only; no executable code or hardware result supplied.

Validation / result: 1/1 graded correct.

Phase Estimation II

Question ID: Phase Estimation II

Problem statement: Let $U=X$. Find the smallest t needed to exactly specify ϕ ; use $|-\rangle$ for $|\psi\rangle$ and find the first-register state after inverse QFT.

Given information: $X|-\rangle = -|-\rangle$; $-1 = e^{(2\pi i \cdot 1/2)}$; binary phase 0.1.

Required result: $t_{\min}$ and the post-inverse-QFT first-register state.

Solution / reasoning: Identify the eigenphase and express it exactly in binary.

Mathematical derivation: $\phi = 1/2 = 0.1_2$, so one bit suffices: $t_{\min} = 1$. The first register is $|-\rangle$ before inverse QFT and $H|-\rangle = |1\rangle$ after it.

Algorithm / procedure: One-qubit QPE / inverse QFT.

Key insight: Demonstrates exact recovery when the phase terminates in one binary digit.

Assumptions: Exact one-bit phase.

Complexity: No implementation benchmark supplied.

Implementation clues: Symbolic/circuit-level implementation evidence only; no executable code or hardware result supplied.

Validation / result: 2/2 graded correct.

Phase Estimation III

Question ID: Phase Estimation III

Problem statement: Let $U=S$. Find the smallest t needed to exactly specify ϕ ; use $|1\rangle$ for $|\psi\rangle$ and find the first-register state after inverse QFT.

Given information: $S|1\rangle = i|1\rangle$; $i = e^{(2\pi i \cdot 1/4)}$; binary phase 0.01.

Required result: $t_{\min}$ and the post-inverse-QFT first-register state.

Solution / reasoning: Identify the eigenphase and represent it exactly in binary.

Mathematical derivation: $\phi = 1/4 = 0.01_2$, so $t_{\min} = 2$. The inverse QFT maps the phase-encoded first register to $|01\rangle$.

Algorithm / procedure: Two-qubit QPE / inverse QFT.

Key insight: Shows that required register precision corresponds to the number of exact binary phase digits.

Assumptions: Exact two-bit phase.

Complexity: No implementation benchmark supplied.

Implementation clues: Symbolic/circuit-level implementation evidence only; no executable code or hardware result supplied.

Validation / result: 2/2 graded correct.

8. Notebook / Computational Asset Record

Notebook name: None supplied for SU3.

Course / problem-set relationship: No executable notebook was supplied; the problem-set evidence was worked from course definitions and circuit diagrams.

Original source: SU3 QFT and phase-estimation problem-set material supplied in the study session.

Language: Not applicable.

Framework / SDK: Not supplied.

Dependencies: QFT/inverse QFT, controlled-unitary synthesis, quantum execution backend, classical reconstruction/validation.

Input: Symbolic quantum states, QFT definition, phase-estimation circuit, and unitary/eigenstate definitions.

Output: Exact transformed states, phase-register bit strings, and conceptual QPE/QFT results.

Algorithm / model: QFT, inverse QFT, phase estimation.

Execution environment: Educational course environment; no external execution environment supplied.

Result: [Enter source-supported information]

Validation: Grader results visible in the supplied screenshots: 3/3, 1/1, 1/1, 1/1, 2/2, 2/2.

Reusable pattern: Symbolic state transformation → phase encoding → inverse QFT → measurable binary representation.

QAI relationship: Candidate evidence for QAI representations of states/operators and hybrid quantum-classical phase estimation; not an implementation artifact.

9. Algorithms and Mathematical Models

Record only what is supported by course, problem-set, solution, notebook, or experiment evidence.

Algorithm: Quantum Fourier Transform; inverse QFT; Quantum Phase Estimation; modular order/period finding as the Shor application context.

Mathematical model — Yes — QFT, inverse QFT, eigenphase and modular-period models.

Inputs: Quantum state/eigenstate, unitary U, precision t ; in Shor, modular multiplication unitary derived from a and N .

Outputs: Measured phase estimate; in Shor, candidate order r and ultimately factors after classical processing.

Complexity: [Enter source-supported information]

Implementation evidence — Limited — circuit diagrams and symbolic implementation structure only.

Related problems: QFT I–III; Phase Estimation I–III; lecture questions on modular exponentiation and factoring using phase estimation.

Related notebooks: None supplied.

10. Extracted Pattern

Pattern — Yes — phase-to-measurement and quantum-plus-classical reconstruction.

Source course / module: 8.370.2x — SU3: The QFT and Shor's quantum factoring algorithm

Source problems: QFT I–III and Phase Estimation I–III; lecture questions on order finding and phase estimation.

Problem-solving structure: Prepare state/eigenstate → controlled powers of a unitary → encode eigenphase by phase kickback → inverse QFT → measure binary phase information → classical reconstruction/validation.

Inputs: [Enter source-supported information]

Outputs: [Enter source-supported information]

Control / feedback: Classical validation determines whether the measured phase yields a valid r ; Shor can retry with another or another quantum sample when reconstruction/factor extraction fails.

Recurring evidence: QFT II/III establishes phase-factor and circuit structure; QPE I–III demonstrates phase encoding and exact binary recovery; lecture questions connect QPE to order finding and Shor.

Potential reuse: Quantum spectral/phase estimation, order finding, periodicity detection, and hybrid quantum-classical algorithms.

Confidence / evidence level: High for the educational algorithmic pattern; medium for broader QAI reuse because no independent implementation evidence was supplied.

11. Primitive Candidate

Primitive name: Phase Estimation / Quantum Phase Extraction Primitive (candidate)

Source evidence: SU3 QFT I–III and Phase Estimation I–III plus lecture questions linking QPE to order finding and Shor.

Pattern: Controlled-U phase encoding → inverse QFT → measurement → phase reconstruction.

Purpose: Estimate an eigenphase of a unitary and expose it as a classical bit string for downstream inference.

Proposed inputs: Unitary U or controlled- U implementation; eigenstate preparation; precision t ; execution/measurement configuration.

Proposed outputs: Measured phase bit string, estimated phase, confidence/reconstruction metadata, and optional candidate order.

API concept: `phase_estimate(U, eigenstate, precision, backend, shots) → phase_estimate_result`.

Runtime requirements: Controlled powers of U , coherent quantum execution, inverse QFT, measurement, and classical phase reconstruction.

Dependencies: [Enter source-supported information]

Validation required: Independent phase-estimation examples, simulator tests, noise sensitivity, resource scaling, and hardware/QPU validation before promotion.

Maturity: Educational candidate; no production implementation evidence.

Promotion recommendation: Retain as a primitive candidate and validate independently; do not promote to the Enterprise Library as a production capability from this SU3 evidence alone.

12. QAI Architecture Mapping

- QAI Language — Represent quantum states, unitary operators, phases, QFT/QPE circuits, measurements, and phase-estimation results symbolically. QAI interpretation layer.
- QAI Primitives — Candidate: phase estimation / quantum phase extraction; supporting QFT/inverse-QFT and controlled-unitary operations.

- QAI OS — Potential orchestration of quantum/classical workflow, retries, policy, and resource selection; proposed, not directly evidenced.
- QAI Runtime — Potential execution of controlled-U, inverse QFT, measurement, and classical reconstruction as a hybrid workflow.
- Hybrid Runtime — Strong conceptual mapping: quantum phase estimation followed by classical continued fractions/GCD validation in Shor.
- QAI Control Plane — Potential control of precision, shots, backend selection, validation, retry budget, and classical post-processing; proposed.
- Quantum Control Plane — Potential control of coherent circuit execution, controlled powers, inverse QFT, and measurement; proposed.
- Adaptive Network Fabric — No direct SU3 evidence; potential transport/resource layer only.
- Resource Registry — Could register QPU/simulator resources and capabilities required by QPE; proposed.
- Capability Registry — Could register QFT, controlled-U, phase-estimation, modular arithmetic and reconstruction capabilities; proposed.
- QAI Labs — Strong fit as an educational/experimental validation area for QFT/QPE and Shor.
- QAI Product Foundry — Potential future design input for reusable quantum algorithm services; not a production capability established by SU3.

13. Classification

- Educational knowledge — Yes — primary evidence category.
- Research — Algorithmic background; no new research result established by this module.
- Mathematical model
- Algorithm
- Pattern
- Prototype — No — no executable SU3 prototype supplied.
- Implementation evidence
- Reusable capability — Candidate only — phase estimation/QFT capability requires independent validation.
- Future capability — Yes — potential QAI hybrid phase/spectral estimation service.

14. Evidence and Provenance

MIT OLL source URL(s): Not supplied in the inspected evidence package.

Original document / problem-set reference: SU3 problem-set screenshots supplied in the study session: Quantum Fourier Transform I–III and Phase Estimation I–III.

Notebook reference: None supplied.

Solution reference: Graded results shown in supplied screenshots; all six problem-set parts were marked correct.

External reference (if any): The lecture concept question included links to Wikipedia pages for quantum phase estimation, Simon's problem, and Shor's algorithm. These were not used as primary evidence.

Evidence limitations: The inspected attachments did not include the original SU3 MIT OLL page hierarchy, lecture URLs, notebook/code, or an independent implementation. QAI mappings and primitive proposals are interpretation, not MIT source claims.

15. Cross-Course Convergence

Identify whether the same concept, algorithm, pattern, or primitive appears in other MIT courses.

Course	Topic / Problem	Common Pattern	Primitive Candidate	Evidence	Status
8.370.2x SU2	Deutsch–Jozsa	Quantum	Quantum +	Completed SU2	Strong conceptual

	and Simon's algorithms; hidden-structure discovery	execution → measurement → classical inference	Classical Solver / Hidden-Structure Discovery	sample supplied	convergence
8.370.2x SU2	Simon hidden-period / Abelian HSP	Hidden periodicity and extraction of a classical parameter from quantum measurements	Hidden-Structure / Constraint Extraction	SU2 sample explicitly connects toward Shor-type HSP	Conceptual link; SU3-specific validation still required
8.370.2x SU3	QFT / phase estimation / Shor	Phase encoding → inverse QFT → measurement → classical reconstruction	Phase Estimation / Quantum Phase Extraction	SU3 problem-set evidence	Current module

16. Reuse and Implementation Assessment

- Direct reuse — No — educational circuit/math is not a production component.
- Pattern reuse — Yes — strong candidate for phase estimation and hybrid quantum-classical workflows.
- Design reference — Yes — QFT/QPE circuit structure and modular arithmetic organization.
- Research input — Yes — phase estimation/order finding as algorithmic research input.
- Architecture input — Yes — supports separation of quantum execution from classical inference/validation.
- Prototype candidate — Yes — suitable for a simulator prototype.
- New implementation required — Yes — any QAI primitive/runtime service would require new implementation and validation.
- Historical / educational reference only — Yes, until independently validated as a reusable capability.

17. Validation Plan

Reference implementation needed: Yes. Implement QFT, inverse QFT, QPE, modular multiplication/exponentiation, and a small Shor factoring example.

Simulator validation: Required. Validate exact phases for X and S examples, QFT basis/superposition examples, and small-N order finding.

Hardware / QPU validation: Required before any production claim. Test QPE/QFT under realistic gate errors and measurement noise.

Independent problem validation: Required. Use additional eigenphase values, larger t, non-exact binary phases, and multiple Shor choices of a.

Cross-course validation: Compare with SU2 hidden-structure patterns and later MIT course modules when inspected.

Benchmark: Phase-estimation accuracy versus qubit precision/shots; circuit depth and controlled-U cost; success probability for small-N Shor examples; retry rate.

Acceptance criteria: Exact toy examples reproduce expected phases and bit strings; small-N Shor cases return correct non-trivial factors with validated probability/retry behavior; no unsupported production-maturity claim.

18. Final Course Summary

- Major concepts — QFT, inverse QFT, tensor-product decomposition, controlled phase, phase estimation, eigenphases, modular exponentiation, order/period finding, Shor factoring, continued fractions, GCD post-processing.
- Important algorithms — Quantum Fourier Transform, inverse QFT, Quantum Phase Estimation, modular exponentiation by repeated squaring, quantum order finding, Shor's factoring workflow.

- Important problem-solving patterns — Phase encoding → inverse QFT → measurement; quantum sampling → classical reconstruction; reversible arithmetic; quantum + classical solver; candidate → validate → retry.
- Important solved questions — QFT I–III and Phase Estimation I–III; all six graded parts completed correctly.
- Important notebooks — None supplied or inspected.
- Primitive candidates — Phase Estimation / Quantum Phase Extraction; supporting QFT/inverse-QFT and controlled-unitary capabilities.
- Enterprise Library contributions — Evidence-backed algorithmic pattern and primitive candidate; not yet a promoted production primitive.
- QAI architecture contributions — Hybrid Runtime, QAI Runtime, Control Plane, QAI Language, capability/resource registry concepts, and QAI Labs validation opportunities.
- Implementation opportunities — Simulator-based QFT/QPE library, modular arithmetic components, Shor order-finding workflow, phase-estimation service API, validation/retry controller.
- Evidence gaps — MIT source URLs/hierarchy not supplied; no SU3 notebook/code; no independent simulator or QPU results; no cross-course validation beyond the supplied SU2 sample.

19. Extraction Status

Area	Status	Notes
Course structure	Partial / module-level	SU3 module evidence captured; full course hierarchy not supplied in this package.
Modules / subunits	Completed	SU3 identified and documented.
Lectures	Completed for reviewed evidence	4 lecture concept questions captured; exact MIT lecture URLs/titles not supplied.
Problem sets	Completed for reviewed evidence	6 problem-set parts captured; 10/10.
Solved questions	Completed	All six problem-set parts solved; lecture concept answers also reviewed.
Notebooks	Not supplied	No SU3 notebook/code asset provided.
Algorithms	Completed	QFT, inverse QFT, QPE, order finding, Shor workflow.
Mathematical models	Completed	QFT and eigenphase models plus modular-period relation.
Patterns	Completed — candidate	Phase-to-measurement and quantum/classical reconstruction pattern extracted.
Primitive candidates	Candidate	Phase estimation / quantum phase extraction proposed; validation required.
QAI mapping	Completed — interpretation layer	Architecture mapping separated from MIT evidence.
Cross-course mapping	Initial	SU2 convergence captured; broader seven-course convergence pending.
Validation	Defined	Simulator, independent examples, hardware and cross-course validation plan recorded.
Final review	Completed for SU3 evidence package	Evidence gaps explicitly recorded.

20. Extraction Rules

- Preserve the actual MIT OLL hierarchy — Preserve the actual MIT OLL hierarchy and terminology; QAI interpretation is kept in the extraction/mapping layers.
- Keep original problem statements — Keep original problem statements and solved questions associated; the six reviewed problem-set parts are captured in Section 7.
- Treat solved questions — Treat solved questions as first-class evidence; all six SU3 problem-set parts were graded correct.
- Treat notebooks — Treat notebooks as implementation evidence; none was supplied for SU3.
- Do not infer implementation — Do not infer implementation maturity from educational material.

- Separate source evidence — Separate source evidence from QAI interpretation; architecture and primitive mappings are explicitly candidates.
- Promote patterns — Promote patterns to primitive candidates only when supported by evidence; SU3 phase estimation remains a candidate pending validation.
- Promote primitives — Promote primitives toward the Enterprise Library only after appropriate validation.
- Use cross-course recurrence — Use cross-course recurrence to strengthen, not replace, validation; SU2 provides the initial convergence reference.

Extraction completion note: SU3 evidence package completed from the supplied study-session material. This record follows the supplied process: preserve → extract → analyze → generalize → validate → map → promote.

QAI Learning Record | MIT 8.370.2x | SU4

SU4: Quantum search and quantum simulation algorithms

Standard Learning & QAI Stack Extraction | Completed

1. Module Overview

SU4 develops Grover's quantum search algorithm and quantum simulation algorithms. The completed study covered the Grover iteration operator, oracle and diffusion/reflection structure, search complexity, small two-qubit examples, Hamiltonian simulation, quantum chemistry simulation, variational quantum eigensolver (VQE), quantum phase estimation (QPE), and continuous-time formulations of quantum search.

The central course-derived engineering lesson is that quantum search and simulation can be represented through different mathematical/circuit forms. The QAI interpretation explored in this session is to preserve higher-level computational intent and select an appropriate representation, synthesis strategy and execution path.

QAI architectural interpretation is intentionally separated from MIT source evidence. Candidate primitives and architecture ideas below are research/design inputs, not claims that the MIT course defines or validates.

2. Core Learning Objectives

- Understand Grover's iteration operator $G = (2|\psi\rangle\langle\psi| - I)O$ as an oracle followed by inversion about the mean/reflection.
- Understand that $|\psi\rangle$ is the equal superposition over computational-basis states and that the oracle marks the target through a phase change.
- Understand the $O(\sqrt{N/M})$ scaling and why the number of marked solutions M matters.
- Track amplitudes and probabilities through the $N=4, M=1$ example; one Grover iteration reaches the marked state with probability 1, while an additional iteration can move away from the optimum.
- Understand quantum simulation as Hamiltonian evolution and the practical cost of Hamiltonian decomposition, including Trotterization.
- Understand quantum chemistry simulation requirements, including fermion-to-qubit transformations such as Jordan–Wigner and Bravyi–Kitaev.
- Distinguish QPE and VQE as approaches to obtaining Hamiltonian eigenvalue/energy information, including the hybrid classical/quantum nature of VQE.
- Understand continuous-time quantum search as Hamiltonian evolution e^{-iHt} , including the Bloch-vector/rotation interpretation and the role of Δt .
- Recognize discrete circuit evolution and continuous Hamiltonian evolution as alternative representations of quantum search.

3. Key Mathematical Models

Model	Expression	Role
Grover operator	$G = (2 \psi\rangle\langle\psi - I)O$	Oracle phase marking followed by inversion about the mean.

Search complexity	$O(\sqrt{N/M})$	Grover scaling for N candidates and M marked solutions.
Equal superposition	$ \psi\rangle = (1/\sqrt{N}) \sum_x x\rangle$	Initial search state for the standard construction.
Hamiltonian simulation	$U(t) = e^{(-iHt)}$	Continuous-time evolution generated by H.
Hamiltonian decomposition	$e^{(-i(H_1+H_2)t)} \approx [e^{(-iH_1t/r)}e^{(-iH_2t/r)}]^r$	Trotter-style approximation; decomposition overhead is part of simulation cost.
Projector reflection	$e^{(-i\pi \psi\rangle\langle\psi)} = I - 2 \psi\rangle\langle\psi $ (up to global phase)	Continuous evolution at $\Delta t=\pi$ reproduces a reflection.
Continuous-time search	$H = x\rangle\langle x + \psi\rangle\langle\psi $	Two-dimensional subspace model used in the continuous-time search problem.
Bloch-vector form	$ x\rangle\langle x = (I + \hat{x} \cdot \sigma)/2$; $ \psi\rangle\langle\psi = (I + \hat{\psi} \cdot \sigma)/2$	Geometric representation used to analyze the continuous-time rotation.
Quantum chemistry Hamiltonian	H_{QC}	Fermionic Hamiltonian transformed into a qubit-compatible representation before standard qubit simulation.

4. Quantum Circuit / Simulation Primitives

- Oracle O — marks a search solution through a phase transformation.
- Diffusion/reflection $D = 2|\psi\rangle\langle\psi| - I$ — inversion-about-the-mean component of Grover.
- Grover operator $G = D O$ — reusable search iteration.
- Hadamard layer $H^{\otimes n}$ — prepares the equal superposition used by standard Grover search.
- Controlled-X/CNOT and X/H gate sequences — concrete circuit elements used to synthesize the two-qubit diffusion/reflection structure.
- Hamiltonian H — logical model for continuous evolution and quantum simulation.
- Time-evolution operator $U(t)=e^{(-iHt)}$ — continuous simulation primitive.
- Trotterization/product-formula decomposition — approximate Hamiltonian evolution when H is a sum of components.
- QPE — phase/eigenvalue estimation mechanism applicable to Hamiltonian simulation.
- VQE — hybrid classical optimization plus quantum subroutine for variational energy estimation.
- Fermion-to-qubit mapping — representation transformation required for standard qubit-based quantum chemistry simulation.

5. Grover Search Algorithm and Solved Problem-Set Learning

The two-qubit case used $N=4$, $M=1$ with marked state $|01\rangle$. The first oracle changes only the phase of $|01\rangle$, producing $(|00\rangle - |01\rangle + |10\rangle + |11\rangle)/2$. After one complete Grover iteration, the state is $|01\rangle$ and the success probability is 1. A second iteration gives probability 1/4, illustrating the need for an optimal stopping/iteration count.

The continuous-time problem set represented search as Hamiltonian evolution, analyzed the corresponding Bloch-vector rotation, identified $\Delta t=\pi$ as the smallest time step reproducing the reflection operations used in the circuit realization, and derived a time-step choice for exact arrival at the solution after $K=O(\sqrt{N})$ rotations.

Important interpretation: Grover is not simply an unlimited iterative loop. The iteration/time parameter is an algorithmic control variable, and overshooting the optimum can reduce success probability.

6. Problem Set Record

Problem set	Questions	Evidence captured	Status
PS#4: Quantum search and quantum simulation	5 graded questions (MIT course map)	Lecture/problem-set screenshots and solved multipart questions covered in this study session	Completed

The MIT course map lists U2.6 Grover's Algorithm (2 questions), U4.6 Quantum simulation algorithms (3 questions), and PS#4 Quantum search and quantum simulation (5 questions).

7. Solved Question Extraction

Q1 – Grover for two qubits

Given information: $N=4$, $M=1$; marked state $|01\rangle$.

Required result: Construct/track the state through the oracle and Grover operator and evaluate measurement probabilities.

Solution / reasoning: Oracle produces a phase flip on $|01\rangle$. One Grover iteration produces $|01\rangle$ with probability 1; a second iteration produces probability $1/4$.

Key insight: Amplitude/phase tracking; reflection about the mean; optimal iteration count.

Validation / result: Validated in the graded result shown in the supplied screenshot.

Q2 – Continuous-time simulation I

Given information: $H=|x\rangle\langle x|+|\psi\rangle\langle\psi|$; $U_x(\Delta t)=e^{-i|x\rangle\langle x|\Delta t}$; $U_\psi(\Delta t)=e^{-i|\psi\rangle\langle\psi|\Delta t}$.

Required result: Complete the Bloch-vector expression for $U(\Delta t)=U_\psi(\Delta t)U_x(\Delta t)$.

Solution / reasoning: Coefficients use $\cos^2(\Delta t/2)$, $\sin^2(\Delta t/2)$, $\sin(\Delta t/2)$, with $\vec{\psi}+\hat{z}$ and $\vec{\psi}\times\hat{z}$.

Key insight: Hamiltonian evolution acts in the two-dimensional subspace spanned by $|x\rangle$ and $|\psi\rangle$.

Validation / result: Validated in the graded result.

Q3 – Continuous-time simulation II

Given information: Interpret $U(\Delta t)$ as a rotation around an axis $\vec{r}$ through angle θ .

Required result: Complete $\vec{r}$ and θ .

Solution / reasoning: $\vec{r} = \cos(\Delta t/2)(\vec{\psi}+\hat{z})/2 + \sin(\Delta t/2)(\vec{\psi}\times\hat{z})/2$; $\theta = 2 \cos^{-1}[1-2 \sin^2(\Delta t/2)/N]$.

Key insight: Continuous-time search has a geometric rotation interpretation.

Validation / result: Validated in the graded result.

Q4 – Continuous-time simulation III

Given information: Find the smallest Δt for which U_ψ and U_x match the circuit-model reflection operations.

Required result: Find Δt and both operators.

Solution / reasoning: $\Delta t=\pi$; $U_\psi(\pi)=I-2|\psi\rangle\langle\psi|$; $U_x(\pi)=I-2|x\rangle\langle x|$.

Key insight: Continuous Hamiltonian evolution can reproduce discrete reflection operations at a selected time step.

Validation / result: Validated in the graded result.

Q5 – Continuous-time simulation IV

Given information: $K=O(\sqrt{N})$ rotations; choose Δt so the final state $|x\rangle$ is obtained exactly.

Required result: Derive the time-step expression using the rotation-angle approximation.

Solution / reasoning: The time step is selected from the required total rotation divided across K rotations, yielding exact arrival in the course construction.

Key insight: Time discretization becomes an algorithmic control parameter; exact final-state arrival is possible in this continuous-time construction.

Validation / result: Validated in the graded result.

8. Notebook / Computational Asset Record

- Notebook name: Not supplied.
- Framework / SDK: Not supplied.
- Execution environment: Not supplied.
- Implementation evidence: Educational problem-set evidence only.
- QAI relationship: create simulator-based reference implementations later for candidate-primitive validation.

9. Algorithms and Mathematical Models

Algorithm/model	Structure	Inputs	Outputs	Complexity / scaling	Evidence
Grover quantum search	Oracle + diffusion/reflection; repeated $O(\sqrt{N/M})$ times.	N, M, oracle	Marked state / measurement result	$O(\sqrt{N/M})$ iterations	Course/problem-set evidence
Hamiltonian simulation	$U(t)=e^{(-iHt)}$ and product-formula/Trotter-style decomposition for sums of Hamiltonians.	H, t, decomposition parameters	Approximate quantum evolution	Depends on decomposition and accuracy	Course lecture/problem-set evidence
Quantum chemistry simulation	Fermionic H_{QC} transformed to a qubit representation; QPE or VQE can estimate energy/eigenvalues.	Hamiltonian, mapping, quantum resources	Eigenvalue/energy estimates	Resource requirements discussed qualitatively	Course lecture/problem-set evidence
Continuous-time Grover search	$H= x\rangle\langle x + \psi\rangle\langle\psi $; evolution interpreted as a rotation.	N, initial state, target, $\Delta t/K$	Solution state / success probability	$K=O(\sqrt{N})$ rotations	Problem-set evidence

10. Extracted Patterns

Amplitude-Amplification / Reflection Pattern

Problem-solving structure: Prepare superposition $\rightarrow$ oracle phase mark $\rightarrow$ reflection $\rightarrow$ repeat $\rightarrow$ measure.

Inputs: Search space N, solution count M, oracle.

Outputs: Marked state.

Control / feedback: Iteration count and measurement/termination policy.

Recurring evidence: High within SU4; cross-course validation recommended.

Potential reuse: QAI algorithm/compiler/runtime.

Confidence / evidence level: Course-supported.

Logical-Representation Transformation Pattern

Problem-solving structure: Represent the same computational intent as circuit, unitary, Hamiltonian or geometric evolution.

Inputs: Logical operation/algorithm + target representation + constraints.

Outputs: Equivalent or approximate representation.

Control / feedback: Select representation according to accuracy/resource/hardware constraints.

Recurring evidence: QAI interpretation inspired by SU4; not an MIT claim.

Potential reuse: QAI Language + Compiler.

Confidence / evidence level: Research candidate.

Hamiltonian Decomposition Pattern

Problem-solving structure: $H = \sum H_i \rightarrow$ decomposition/product formula $\rightarrow$ approximate evolution.

Inputs: Hamiltonian terms, time, error tolerance.

Outputs: Implementable evolution sequence.

Control / feedback: Approximation error vs resource cost.

Recurring evidence: Directly supported by SU4 lecture question.

Potential reuse: QAI Compiler/Runtime.

Confidence / evidence level: Course-supported.

Hybrid Quantum-Classical Pattern

Problem-solving structure: Quantum subroutine + classical computation/optimization $\rightarrow$ updated parameters/result.

Inputs: Quantum circuit/Hamiltonian + classical optimizer.

Outputs: Energy/eigenvalue or optimized parameters.

Control / feedback: Classical/quantum loop control.

Recurring evidence: Directly supported by VQE discussion.

Potential reuse: Hybrid Runtime.

Confidence / evidence level: Course-supported.

Resource-Aware Search/Simulation Control Pattern

Problem-solving structure: Choose iteration/time-step based on problem size and target success/fidelity under resource constraints.

Inputs: N , M , target probability, $\Delta t/K$, resource budget.

Outputs: Execution/stopping policy.

Control / feedback: Iteration/time is an explicit control variable.

Recurring evidence: QAI interpretation; validation required.

Potential reuse: QAI Control Plane.

Confidence / evidence level: Research candidate.

11. Primitive Candidates

11. Primitive Candidates

QAI_GroverSearch

Source evidence: Grover lectures + PS#4

Pattern: Amplitude amplification pattern

Purpose: Execute or synthesize a Grover search for N and M.

Proposed inputs: N, M, oracle, target success probability, backend constraints

Proposed outputs: Search result / execution plan

API concept: QAI Compiler + Runtime

Runtime requirements: Oracle implementation; reflection synthesis; backend mapping

Dependencies: Oracle implementation; reflection synthesis; backend mapping.

Validation required: Multiple N/M cases, predicates and backends.

Maturity: Candidate

Promotion recommendation: Validate independently before Enterprise Library promotion.

QAI_Oracle

Source evidence: Grover oracle construction

Pattern: Phase-marking pattern

Purpose: Represent a logical search predicate as a reversible/phase oracle.

Proposed inputs: Predicate, target register, ancilla/workspace, phase convention

Proposed outputs: Logical oracle / circuit

API concept: QAI Language + Compiler

Runtime requirements: Reversible synthesis and hardware mapping

Dependencies: Reversible predicate/function synthesis; ancilla/workspace handling.

Validation required: Multiple predicates and reversible implementations.

Maturity: Candidate

Promotion recommendation: Validate with multiple predicates and backends.

QAI_Reflection

Source evidence: Grover diffusion/reflection

Pattern: Reflection about a known state/subspace

Purpose: Synthesize $I - 2|\psi\rangle\langle\psi|$ or equivalent logical reflection.

Proposed inputs: Reference state/subspace, target register

Proposed outputs: Logical reflection circuit/operator

API concept: QAI Compiler

Runtime requirements: Gate synthesis and native mapping

Dependencies: State preparation; gate synthesis; backend mapping.

Validation required: Compare logical reflection with synthesized circuits.

Maturity: Candidate

Promotion recommendation: Benchmark depth, ancilla and fidelity.

QAI_HamiltonianEvolution

Source evidence: Continuous-time simulation

Pattern: $U(t)=e^{(-iHt)}$

Purpose: Represent and execute Hamiltonian evolution under resource constraints.

Proposed inputs: H, t, accuracy, backend

Proposed outputs: Evolution operator/circuit

API concept: QAI Runtime + Quantum Control Plane

Runtime requirements: Hamiltonian decomposition/simulation method

Dependencies: Hamiltonian representation; evolution/simulation backend.

Validation required: Simulator and hardware comparison across Hamiltonians.

Maturity: Candidate

Promotion recommendation: Simulator + hardware validation.

QAI_HamiltonianDecomposition

Source evidence: Hamiltonian simulation lecture

Pattern: $H=\sum H_i \rightarrow$ product formula

Purpose: Choose/decompose Hamiltonian terms for implementable evolution.

Proposed inputs: H terms, time, error tolerance

Proposed outputs: Decomposition schedule

API concept: QAI Compiler

Runtime requirements: Trotter/product-formula implementation

Dependencies: Hamiltonian term representation; product-formula/Trotter method.

Validation required: Compare approximation error, depth and runtime.

Maturity: Candidate

Promotion recommendation: Compare approximation error and cost.

QAI_RepresentationTransform

Source evidence: QAI architectural interpretation from SU4

Pattern: Circuit $\leftrightarrow$ unitary $\leftrightarrow$ Hamiltonian $\leftrightarrow$ state/geometric representations

Purpose: Preserve logical intent while changing representation.

Proposed inputs: Logical algorithm/object, target representation, constraints

Proposed outputs: Equivalent/approximate logical representation

API concept: QAI Language + Compiler

Runtime requirements: Semantic equivalence and approximation validation

Dependencies: Semantic equivalence model; circuit/unitary/Hamiltonian/state representations.

Validation required: Cross-representation functional equivalence and approximation tests.

Maturity: Research candidate

Promotion recommendation: Cross-course validation required.

QAI_SearchExecutionPolicy

Source evidence: QAI interpretation

Pattern: Optimal iteration/time-step and termination control

Purpose: Select execution length based on N, M, accuracy and resource constraints.

Proposed inputs: N, M, target probability, $\Delta t/K$, resource budget

Proposed outputs: Execution/stopping policy

API concept: QAI Control Plane

Runtime requirements: Runtime measurement and feedback

Dependencies: Search model; measurement feedback; resource profile.

Validation required: Discrete and continuous search benchmarks under resource limits.

Maturity: Research candidate

Promotion recommendation: Validate against discrete and continuous search variants.

QAI_QuantumObjectLifecycle

Source evidence: QAI architectural discussion

Pattern: Create $\rightarrow$ compose $\rightarrow$ manipulate $\rightarrow$ validate $\rightarrow$ execute $\rightarrow$ measure $\rightarrow$ destroy

Purpose: Manage logical quantum structures such as qubits, ancillas, oracles and circuits.

Proposed inputs: Object type, lifecycle policy, logical/physical mapping

Proposed outputs: Lifecycle state and control actions

API concept: QAI Language + Quantum Control Plane

Runtime requirements: Resource registry and physical allocation

Dependencies: Resource registry; logical/physical mapping; lifecycle controller.

Validation required: Broader architecture, simulator and hardware/resource validation.

Maturity: Future capability

Promotion recommendation: Requires broader architectural and hardware validation.

12. QAI Architecture Mapping

QAI layer	SU4 contribution / interpretation
QAI Language	First-class logical objects: search problem, oracle, reflection, circuit, Hamiltonian, time evolution, state/geometry, measurement and lifecycle metadata.
QAI Primitives	Grover, oracle, reflection, Hamiltonian evolution/decomposition, representation transformation and resource-aware search control candidates.
QAI OS	Resource and lifecycle management for logical/virtual/physical quantum assets; scheduling and policy separation.
QAI Runtime	Discrete circuit execution, Hamiltonian evolution/simulation and coordination with classical computation.
Hybrid Runtime	VQE-style classical optimization plus quantum subroutines; adaptive execution and parameter updates.
QAI Control Plane	Iteration count, time-step selection, measurement/termination policy, accuracy/resource constraints and orchestration.
Quantum Control Plane	Logical-to-physical mapping, native gate/control realization, QEC/FTQC interfaces and hardware feedback.
Adaptive Network Fabric	Potential future transport/control abstraction for distributed quantum resources; not established by SU4 evidence.
Resource Registry	Candidate registry for qubits, ancillas, circuits, Hamiltonians, simulators and physical backend resources.
Capability Registry	Mapping/synthesis capabilities, native gates, Hamiltonian simulation methods and backend constraints.
QAI Labs	Reference implementations and simulator/hardware validation of candidate primitives.
QAI Product Foundry	Potential future engineering path from validated primitives to reusable QAI services/products.

13. Classification

- Educational knowledge — Grover search, quantum simulation, quantum chemistry simulation, QPE, VQE, continuous-time search.
- Mathematical model — Grover operator, projectors/reflections, Hamiltonian evolution, Bloch-vector rotation.
- Algorithm — Grover search, Hamiltonian simulation approaches, QPE/VQE as discussed in the course.
- Pattern — amplitude amplification, Hamiltonian decomposition, hybrid execution, representation transformation (QAI interpretation).
- Prototype / implementation evidence — not supplied in this evidence package.
- Reusable capability — candidate only; validation is required.
- Future capability — logical-object lifecycle, representation optimization, coherence/resource-aware execution and FTQC-oriented control integration.

14. Evidence and Provenance

- MIT source: MIT Open Learning Library, 8.370.2x Quantum Information Science I, Part 2.
- Official course page: <https://openlearninglibrary.mit.edu/courses/course-v1:MITx+8.370.2x+1T2018/course/>
- Official syllabus: <https://openlearninglibrary.mit.edu/courses/course-v1:MITx+8.370.2x+1T2018/syllabus/>
- Official course map identifies SU4 as 'Quantum search and quantum simulation algorithms', with U2.6 (2 questions), U4.6 (3 questions), and PS#4 (5 questions).
- Primary evidence in this package: supplied screenshots and solved question interactions from the SU4 study session.

- Evidence limitations: no notebook/code or independent hardware experiment was supplied for SU4; QAI primitive candidates therefore remain candidates until independently implemented and validated.

15. Cross-Course Convergence

Course	Topic / problem	Common pattern	Primitive candidate	Evidence	Status
SU2	Deutsch–Jozsa / Simon	Oracle, phase kickback, interference, logical circuit synthesis	QAI_Oracle / QAI_Operation / QAI_Circuit	SU2 extraction record	Strong
SU3	QFT / Shor	Phase, eigenvalue/period information, classical post-processing, synthesis	QAI_RepresentationTransform / spectral-estimation candidates	SU3 study session	Strong conceptual convergence
SU4	Grover / simulation	Oracle, reflections, Hamiltonian evolution, hybrid execution	QAI_GroverSearch / QAI_HamiltonianEvolution	This extraction	Current evidence
Future 8.371x	Fault tolerance / complexity	Logical-to-physical separation, QEC and fault tolerance	FTQC-oriented lifecycle/control primitives	MIT identifies 8.371x as advanced follow-on	Pending future study

16. Reuse and Implementation Assessment

- Direct reuse: course algorithms as educational/reference algorithms only.
- Pattern reuse: oracle, reflection, Hamiltonian evolution, decomposition and hybrid-execution patterns.
- Design reference: representation transformation, logical/physical separation, lifecycle management and resource-aware execution.
- Research input: continuous/discrete representation equivalence, geometry-aware representations and coherence-budget optimization.
- Architecture input: QAI Language, QAI Compiler, Runtime, Control Plane, Quantum Control Plane, Resource/Capability Registries.
- Prototype candidate: simulator implementations of Grover, Hamiltonian evolution, VQE/QPE workflows and representation transforms.
- New implementation required: all QAI-specific primitives and orchestration policies.
- Historical / educational reference only: any claim of production FTQC capability not supported by this evidence package.

17. Validation Plan

- Reference implementation needed: Python/Qiskit or another agreed simulator implementation for each candidate primitive.
- Simulator validation: verify Grover amplitude amplification, Hamiltonian evolution, Trotterization error, QPE/VQE behavior and continuous-time search.
- Hardware / QPU validation: compare logical predictions with native-gate implementations, noise and coherence constraints when suitable hardware is available.
- Independent problem validation: use multiple search predicates, different N/M ratios and multiple Hamiltonians.
- Cross-course validation: compare oracle and representation patterns with SU2 and QFT/phase-estimation patterns from SU3.
- Benchmark: gate count, circuit depth, ancilla count, approximation error, success probability, runtime and—where measurable—energy/resource consumption.
- Acceptance criteria: functional equivalence within defined tolerance; documented resource profile; reproducible results; backend-aware implementation evidence.
- FTQC progression: after primitive validation, integrate logical-qubit/QEC representations and measure whether the abstraction survives mapping to fault-tolerant logical operations.

18. QAI Architecture Extensions Discussed During SU4 Study

- First-class quantum objects: qubits, logical qubits, registers, gates, unitaries, circuits, oracles, ancillas, entanglement structures, phase-kickback operations, measurements and Hamiltonians.
- Quantum-object lifecycle: Create → Compose → Manipulate → Validate → Execute → Measure → Destroy, with static and dynamic forms.
- Logical/physical duality: logical quantum objects may have virtual/digital-twin representations and physical realizations, subject to validation and hardware mapping.
- Representation layer: allow a logical computational intent to be represented as a circuit, unitary, Hamiltonian, state/geometric model or other validated form, with transformations between representations.
- Classical-to-quantum engineering reuse: reuse classical circuit decomposition, modularity, parallelism, resource sharing, critical-path and optimization principles where they can be transformed into reversible/quantum-compatible designs.
- Simulation hierarchy: simulation → emulation → virtualisation/digital twin → physical hardware, with progressively stronger implementation evidence.
- Resource-aware execution: optimize not only gate count but also depth, qubits, ancillas, fidelity, runtime, energy and coherence-related constraints.
- Coherence-aware execution hypothesis: treat T1/T2 and related physical lifetimes as execution constraints analogous in spirit to a time-to-live budget, while avoiding a literal equivalence between TTL and T1/T2.
- Geometry/structured-representation research direction: investigate Bloch geometry, polar/quaternion representations, manifold methods and tensor-network/MPS representations as possible structured computational representations. These are research hypotheses, not SU4 conclusions.
- Continuity-aware computation: investigate whether continuous trajectories and smooth representations can support efficient simulation/inference while respecting finite coherence and resource budgets.
- FTQC progression: use logical-object, representation, lifecycle and control-plane abstractions as a possible foundation for integrating QEC and fault-tolerant logical operations later.

19. Final Subunit Summary

SU4 completed the learning path from Grover's discrete amplitude amplification to quantum simulation and continuous-time quantum search. The subunit demonstrated that search and simulation can be expressed through multiple mathematical and circuit representations: oracle/reflection circuits, Hamiltonian evolution, and Bloch-vector rotations. Quantum chemistry examples added fermion-to-qubit transformations, QPE and VQE, showing how physical models are transformed into computational representations and then processed through quantum or hybrid algorithms.

For the QAI stack, the strongest extracted direction is a separation between logical intent and implementation representation. Candidate abstractions include first-class quantum objects, reusable search/reflection/Hamiltonian primitives, representation transformations, lifecycle management, hybrid runtime control and resource/coherence-aware execution. These are candidate research/engineering capabilities and require independent validation before promotion to the QAI Enterprise Library or any production/FTQC claim.

Long-term architecture hypothesis: Problem → Logical representation → Primitive/algorithm synthesis → Representation transformation → Simulation/emulation/digital twin → Hardware-aware mapping → Physical execution → Measurement/feedback → Validation/QEC/FTQC integration.

20. Extraction Status

Area	Status	Notes
Course structure	Complete for SU4	Official MIT course map verified.
Module / subunit	Complete	SU4 captured.
Lectures	Complete	U2.6 (2 questions) and U4.6 (3 questions) studied.
Problem sets	Complete	PS#4 (5 questions) studied.
Solved questions	Complete	Lecture and PS questions discussed; graded results recorded where shown.
Notebooks	Not supplied	No notebook/code evidence in this package.
Algorithms	Complete	Grover, Hamiltonian simulation, QPE/VQE and continuous-time search captured.

Mathematical models	Complete	Operators, projectors, Hamiltonians and Bloch-vector models captured.
Patterns	Complete / candidate	Source patterns separated from QAI interpretations.
Primitive candidates	Candidate	Validation required before promotion.
QAI mapping	Complete / candidate	Architecture mapping recorded; future layers explicitly marked speculative.
Cross-course mapping	Initial	SU2/SU3 convergence recorded; broader convergence pending.
Validation	Planned	Simulator, hardware, independent and cross-course validation defined.
Final review	Complete	SU4 extraction package completed.

21. Future Reading / Course Reference

- MIT Open Learning Library — 8.370.2x Quantum Information Science I, Part 2; SU4: Quantum search and quantum simulation algorithms.
- Official course page: <https://openlearninglibrary.mit.edu/courses/course-v1:MITx+8.370.2x+1T2018/course/>
- Official syllabus: <https://openlearninglibrary.mit.edu/courses/course-v1:MITx+8.370.2x+1T2018/syllabus/>
- MIT identifies the 8.371x sequence as the advanced follow-on including fault-tolerant quantum computation, advanced quantum communication and advanced quantum algorithms.